

MODULE 11

Neural Networks and Deep Learning

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Neural Networks and Deep Learning

What is a Neural Network?

Neural networks are inspired by the human brain.
Consist of layers of interconnected neurons.

Layers:

Input Layer: receives raw features

Hidden Layers: learn representations

Output Layer: produces predictions

Each neuron: weighted sum of inputs + activation function

$y = \text{activation}(w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + \text{bias})$

Activation Functions

```
import numpy as np

# Sigmoid: 0 to 1 (binary output)
sigmoid = lambda x: 1 / (1 + np.exp(-x))

# ReLU: max(0, x) (most popular in hidden layers)
relu = lambda x: np.maximum(0, x)

# Tanh: -1 to 1
tanh = np.tanh

# Softmax: multi-class probabilities
softmax = lambda x: np.exp(x) / np.exp(x).sum()

# Best practices:
# Hidden layers: ReLU
# Binary output: Sigmoid
# Multi-class: Softmax
```

Build Neural Network with Keras

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization

model = Sequential([
    Dense(128, activation='relu', input_shape=(n_features,)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(1, activation='sigmoid') # binary classification
])
```

```
])
```

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])  
model.summary()
```

Training Neural Networks

```
import tensorflow as tf
```

```
# Callbacks
```

```
early_stop = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=5)
```

```
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5)
```

```
checkpoint = tf.keras.callbacks.ModelCheckpoint('best_model.h5', save_best_only=True)
```

```
history = model.fit(
```

```
X_train, y_train,
```

```
validation_data=(X_val, y_val),
```

```
epochs=100, batch_size=32,
```

```
callbacks=[early_stop, reduce_lr, checkpoint]
```

```
)
```

```
# Plot training history
```

```
import matplotlib.pyplot as plt
```

```
plt.plot(history.history['accuracy'])
```

```
plt.plot(history.history['val_accuracy'])
```

Backpropagation and Optimizers

Backpropagation: gradient descent through layers

1. Forward pass: compute predictions
2. Compute loss
3. Backward pass: compute gradients
4. Update weights: $w = w - lr * \text{gradient}$

Optimizers:

SGD: basic gradient descent

Momentum: accelerates in consistent directions

Adam: adaptive learning rate (most popular)

RMSprop: good for RNNs

Loss Functions:

Binary crossentropy: binary classification

Categorical crossentropy: multi-class

MSE: regression